Reinforcement Learning

Part 3: when the table will not fit

Part 3 of 7. Function approximation, DQN, and the three ingredients that make it diverge.

The gridworld had 16 squares

Tabular Q-learning stored one number per (s, a) pair. For the gridworld that was $16 \times 4 = 64$ numbers. Fine.

Now play Atari from the screen

One frame is 210×160 pixels with 128 possible colours each:

$$|\mathcal{S}| \approx 128^{33{,}600}$$

For scale, there are roughly 10^{80} atoms in the observable universe. This number has more than seventy thousand digits.

There is no table. There will never be a table. And yet almost every state you will ever see *looks like* one you have seen before – which is the opening every other chapter of this course has used.

Outline

Generalise instead of memorise

Why it breaks

DQN

Everything that fixed DQN

Reality check

Replace the table with a function

The move you have made in every chapter since regression.

From a table to a network

Tabular

$Q(s, a)$ is a cell you look up.

Updating one cell teaches you **nothing** about any other cell – even one that differs by a single pixel.

Approximate

$Q_\theta(s, a)$ is a network with a few million weights.

Updating it for one state **moves every similar state with it**. That is the point, not a side effect.

The loss is the squared TD error from lecture 2 – the surprise, squared:

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\big(\underbrace{r + \gamma \max_{a'} Q_{\theta^-}(s', a')}_{\text{target } y} - Q_\theta(s, a)\big)^2\Big]$$

It looks exactly like the regression loss from chapter 1. It is not, and the next two frames are about why.

This is not regression, however much it looks like it

Supervised learning

- Targets y are fixed, given, correct.
- The dataset does not change while you fit it.
- Samples are i.i.d.

Fitting Q_θ

- Targets contain Q_θ itself. They **move when you learn**.
- Better policy $\Rightarrow$ different states visited $\Rightarrow$ different data.
- Consecutive frames are nearly identical.

The semi-gradient

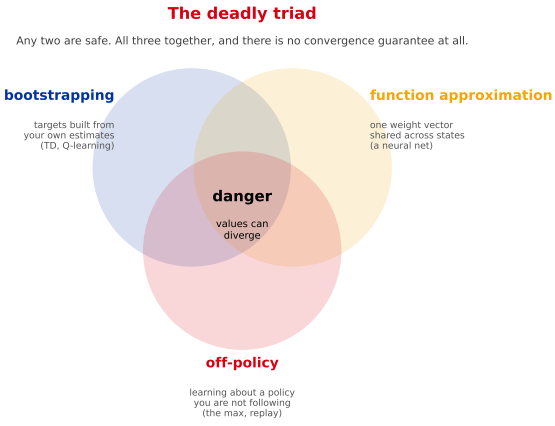
Honesty about the maths: when we differentiate that loss, we treat the target y as a constant even though it depends on θ. So the update is **not** the gradient of any loss function. Sutton calls it a *semi-gradient* method.

It works, it is what everyone does, and it is the reason none of the convergence guarantees from lecture 2 survive.

Three ingredients, one explosion

Each is harmless. Together they have no guarantee at all.

The deadly triad



Drop any **one** and you are safe:

- No bootstrapping $\Rightarrow$ Monte Carlo with a network. Converges, but slow and high variance.
- No approximation $\Rightarrow$ tabular. Converges, but does not scale.
- No off-policy $\Rightarrow$ on-policy methods like SARSA. Converges, but you cannot reuse old data.

DQN wants all three – and values can grow without bound.

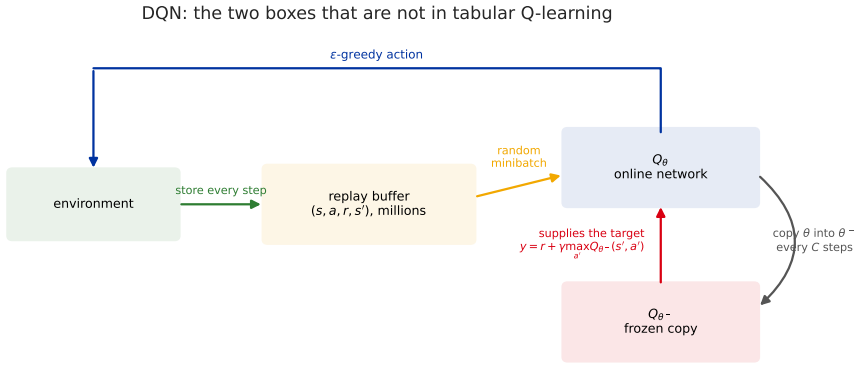
Why the combination is unstable: an over-estimate at s' raises the target for s; the network generalises, so that also raises $Q(s', \cdot)$ – which raises the target again. Nothing pulls it back down.

Two hacks that made it work

Neither is deep. Both are load-bearing.

DQN

The **Deep Q-Network** (**DQN**), Mnih et al., 2015.



Replay breaks the correlation between consecutive frames. The frozen copy stops the network chasing its own output.

Input is four stacked frames (so velocity is visible – one frame cannot tell you which way the ball is moving), through a small convolutional network, out to one Q value per action. **One forward pass scores every action at once.**

Why each hack exists

Experience replay

Store transitions (s, a, r, s') in a buffer of millions and train on *random* batches from it.

Why: consecutive frames are almost identical, so training on them in order violates i.i.d. badly – you would take a hundred gradient steps on essentially the same picture, then a hundred on the next one, and forget the first. Shuffling restores something like an ordinary dataset. *Bonus: each transition gets used many times instead of once.*

Target network

Compute the target with a *frozen* copy Q_{θ^-}, refreshed every C steps.

Why: otherwise every gradient step changes the target you are regressing towards – a dog chasing its own tail. Freezing it turns each interval into an ordinary supervised problem with fixed labels, which is the only reason the deadly triad does not blow up.

Replay is what makes DQN **off-policy** – the batch came from older policies. Q-learning's max is what makes that legal.

What it achieved

One architecture, one set of hyperparameters, learned **49 Atari games from raw pixels**, reaching at least 75% of a professional human tester's score on **29** of them (Mnih et al., *Nature* 2015).

Why this was the shock, and it was not the scores

Before this, game-playing agents were hand-engineered per game – someone wrote features for "ball position" and "paddle position". DQN was given the pixels and the score and nothing else, and the *same* program learned Breakout, Pong and Space Invaders.

That is the bitter lesson in miniature, and it is why the field pivoted.

And the honest part. It failed completely on games needing long-horizon planning. On Montezuma's Revenge – where you must fetch a key, cross a room and open a door before any reward appears – DQN scored **zero**. Sparse reward defeats it entirely.

Six patches and a rainbow

Each one names a specific thing that was wrong.

Double DQN: the bias you already met

Lecture 2 measured it on the gridworld: **13 of 13 states overestimated**, because $\max_{a'} Q(s', a')$ takes a maximum over noisy estimates and the noise gets selected for.

With a network the problem is worse, not better – there is more noise and it is correlated across states.

DQN

$$y = r + \gamma \max_{a'} Q_{\theta^-}(s', a')$$

One network both *picks* the action and *says how good it is*. If it is wrong, it is wrong twice in the same direction.

Double DQN

$$y = r + \gamma\, Q_{\theta^-}\big(s', \arg\max_{a'} Q_\theta(s', a')\big)$$

The online net *chooses*, the frozen net *scores*. The noise cannot select and reward itself.

van Hasselt (2010) for tabular, van Hasselt, Guez & Silver (2016) for the DQN version. **Two networks were already lying around**, which is what makes the fix nearly free.

Four more, each fixing something specific

Extension	The problem	The fix
Dueling (Wang et al., 2016)	In most states the action barely matters, but you must still learn $\lvert\mathcal{A}\rvert$ separate values.	Split the head into $V(s)$ and advantage $A(s,a)$, recombine as $Q = V + (A - \bar{A})$. Learns V from every action.
Prioritised replay (Schaul et al., 2016)	Uniform sampling wastes most batches on transitions the network already predicts perfectly.	Sample in proportion to $\lvert\delta\rvert$, the TD error. *And correct the resulting bias with importance sampling* – lecture 2's ratio, third appearance.
Distributional (Bellemare et al., 2017)	A mean return hides everything: "0 on average" could be certain, or ± 100.	Predict the whole *distribution* of returns instead of its mean (C51).
Noisy nets (Fortunato et al., 2018)	ϵ-greedy explores by twitching randomly, identically in every state.	Put learnable noise in the weights, so exploration is state-dependent and anneals itself.

Rainbow: all of them at once

Hessel et al. (2018) combined **six** extensions – double, dueling, prioritised replay, distributional, noisy nets, and multi-step returns (the n-step dial from lecture 2) – into a single agent that beat every individual variant.

The part worth remembering is the ablation

They removed each component in turn. **Prioritised replay** and **multi-step returns** hurt the most when taken away; **noisy nets** and **dueling** mattered least, and on some games removing them helped.

So it is not six equally good ideas – it is two that carry the result and four that pay their way on some games and not others.

The habit to copy: when a paper stacks N improvements, look for the ablation before you believe in any of them. If there is no ablation, assume one or two components are doing the work.

What the papers do not put on the slide

Sample cost, and the seed problem.

The price nobody puts on the slide

DQN's budget, per game

- 50 million frames
- $\approx$ 38 days of continuous play
- and that is for *one* of the 49 games

A human is competent at Breakout in about **five minutes**.

This gap is the single biggest practical objection to deep RL, and it decides where you can use it at all.

Which is why the simulator matters more than the algorithm

In a simulator, 38 days of play is a few hours on a cluster and a mistake costs nothing. On a physical robot it is 38 days of wear, supervision and broken hardware. On a live recommender it is 38 days of showing users things you know are bad.

Ask "how do I get millions of cheap, safe mistakes?" before asking which algorithm to use. If there is no good answer, the honest move is to not use RL – lecture 5 comes back to this.

The seed problem

Deep RL that Matters (Henderson et al., 2018)

They took single algorithms and ran them with different random seeds. Splitting the runs into two groups and comparing them, **the same algorithm can look significantly better than itself**.

Results also moved with the codebase, the reward scaling, the network size and the activation function – none of which are usually reported as choices.

Why RL is worse than the rest of ML here: the policy generates its own data, so a lucky early episode changes what the agent sees next, which changes what it learns, which changes what it sees. Small differences compound instead of averaging out.

Practically: report multiple seeds with a spread, never a single curve. Treat any single-seed comparison – yours or a paper's – as an anecdote.

If you want to run this yourself

You should not implement DQN from scratch for real work, and you should implement it once for understanding. The ecosystem:

Gymnasium	the environment interface – `env.step(a)`, the maintained successor to G
Stable-Baselines3	solid reference implementations. Use this to get a baseline before writing
CleanRL	single-file implementations, a few hundred lines each. Best for *reading*
Hugging Face Deep RL Course	free, hands-on, Colab notebooks for exactly the algorithms in this lecture

A working DQN on CartPole is about 150 lines. A working DQN on Atari is the same 150 lines plus two weeks of finding out which hyperparameter you got wrong – which is the real lesson of this lecture.

Recap

- A table cannot hold a real state space. Replace it with Q_θ and updates **generalise** to similar states – which is the whole point.
- Fitting Q_θ is not regression: the targets move as you learn, the data distribution moves as the policy improves, and the update is a **semi-gradient** rather than the gradient of anything.
- The **deadly triad** – bootstrapping, function approximation, off-policy – has no convergence guarantee. Any two of the three do.
- **DQN** survives it with two hacks: **experience replay** (breaks correlation, reuses data) and a **target network** (stops the target moving).
- Double, dueling, prioritised replay, distributional and noisy nets each fix one named defect; **Rainbow** stacks them, and its ablation says prioritised replay and multi-step returns do most of the work.
- The costs that decide real projects: **50M frames per game**, and results that move between random seeds.

Next: everything so far learned *values* and acted greedily. What if you skip the values and optimise the behaviour itself? That path leads to PPO – and to every method used on language models.